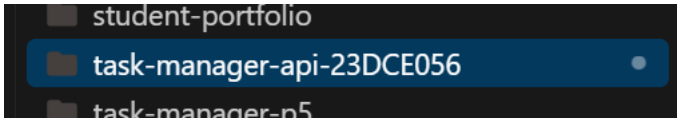
Practical 4: Building a RESTful API with Node.js and Express

Objective	To design and implement a RESTful backend server with complete CRUD endpoints using Express middleware pipeline.
Architecture / Diagram	
Diagram	Client (Postman/Browser) | v [Logging Middleware] | v Express Router ├── GET /tasks → getAllTasks ├── POST /tasks → createTask ├── PUT /tasks/:id → updateTask └── DELETE /tasks/:id → deleteTask | v [Global Error Handler]
Problem Definition	
Problem Statement	• Build a Node/Express backend for a Task Management system. • Implement REST endpoints for creating, reading, updating, and deleting tasks (in-memory array, no database yet). • Apply a request logging middleware that logs method, URL, and timestamp for every incoming request. • Apply a global error handling middleware as the last middleware in the pipeline. • Use correct HTTP status codes (200, 201, 404, 500) for each response.

Steps

1. Initialize a new Node project and install Express:
mkdir task-manager-api && cd task-manager-api
npm init -y
npm install express




```
PS C:\Users\JIGNESH\JIGNESH USER BACKUP DATA\all Important document\Desktop\All-AWDF- Practical> cd task-manager-api-23DCE056
PS C:\Users\JIGNESH\JIGNESH USER BACKUP DATA\all Important document\Desktop\All-AWDF- Practical\task-manager-api-23DCE056> npm init -y
>> npm install express
>>
Wrote to C:\Users\JIGNESH\JIGNESH USER BACKUP DATA\all Important document\Desktop\All-AWDF- Practical\task-manager-api-23DCE056\package.json:

{
  "name": "task-manager-api-23dce056",
  "version": "1.0.0",
  "description": "**Roll Number:** 23DCE056   **Student Name:** Jignesh Katariya   **Course/Branch:** Computer Engineering",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1",
    "start": "node server.js"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "dotenv": "^17.4.2",
    "express": "^5.2.1",
    "mongoose": "^9.9.3"
  },
  "devDependencies": {}
}

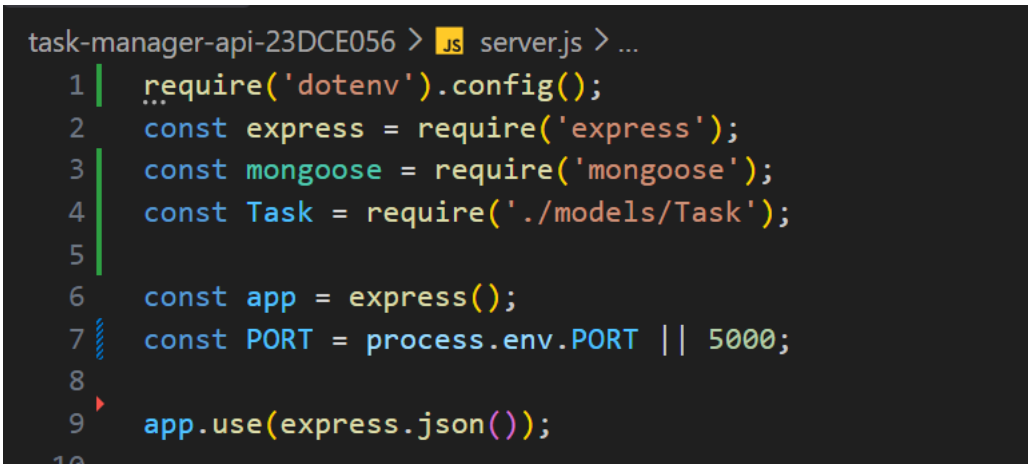

up to date, audited 87 packages in 4s

28 packages are looking for funding
  run `npm fund` for details

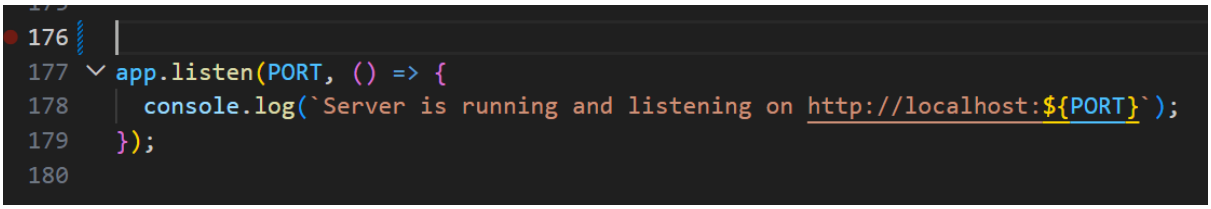
1 moderate severity vulnerability
```

2. Create server.js and set up a basic Express app:

```
const express = require('express');
const app = express();
app.use(express.json());
app.listen(5000, () => console.log('Server running on port
5000'));
```



```
task-manager-api-23DCE056 > server.js > ...
1  require('dotenv').config();
2  const express = require('express');
3  const mongoose = require('mongoose');
4  const Task = require('./models/Task');
5
6  const app = express();
7  const PORT = process.env.PORT || 5000;
8
9  app.use(express.json());
```



```
176
177 app.listen(PORT, () => {
178   console.log(`Server is running and listening on http://localhost:${PORT}`);
179 });
180
```

3. Add a request logging middleware applied globally:

```
app.use((req, res, next) => {
  console.log(`${req.method} ${req.url} - ${new
Date().toISOString()}`);
  next();
});
```

```
const validateTaskId = (req, res, next) => {
  if (!mongoose.Types.ObjectId.isValid(req.params.id)) {
    return res.status(400).json({
      error: "Bad Request: Invalid task ID format. Task ID must be a valid 24-character hexadecimal string."
    });
  }
  next();
};


app.get('/tasks', async (req, res, next) => {
  try {
    const tasks = await Task.find();
    res.status(200).json(tasks);
  } catch (err) {
    next(err);
  }
});


app.get('/tasks/:id', validateTaskId, async (req, res, next) => {
  try {
    const task = await Task.findById(req.params.id);
    if (!task) {
      return res.status(404).json({ error: `Task with ID ${req.params.id} not found.` });
    }
    res.status(200).json(task);
  } catch (err) {
    next(err);
  }
});
```

```
app.post('/tasks', async (req, res, next) => {
  try {
    const { title, description, completed, priority } = req.body;

    const newTask = await Task.create({
      title,
      description,
      completed,
      priority
    });

    res.status(201).json(newTask);
  } catch (err) {
    next(err);
  }
});

app.put('/tasks/:id', validateTaskId, async (req, res, next) => {
  try {
    const updatedTask = await Task.findByIdAndUpdate(
      req.params.id,
      req.body,
      { new: true, runValidators: true }
    );

    if (!updatedTask) {
      return res.status(404).json({ error: `Task with ID ${req.params.id} not found.` });
    }

    res.status(200).json(updatedTask);
  } catch (err) {
    next(err);
  }
});
```

```
app.delete('/tasks/:id', validateTaskId, async (req, res, next) => {
  try {
    const deletedTask = await Task.findByIdAndDelete(req.params.id);

    if (!deletedTask) {
      return res.status(404).json({ error: `Task with ID ${req.params.id} not found.` });
    }

    res.status(200).json({
      message: "Task deleted successfully.",
      task: deletedTask
    });
  } catch (err) {
    next(err);
  }
});

app.get('/trigger-error', (req, res, next) => {
  next(new Error("This is a deliberately thrown server error to test global error handling."));
});
```

4. Implement the 4 CRUD routes using an in-memory array as temporary storage.

```
app.use((req, res, next) => {
  res.status(404).json({ error: `Cannot ${req.method} ${req.url}. Route not found.` });
});
```

5. Add a global error handling middleware as the final middleware:

app.use((err, req, res, next) => {
console.error(err.stack);
res.status(500).json({ error: 'Something went wrong' });
});

```
app.use((req, res, next) => {
  res.status(404).json({ error: `Cannot ${req.method} ${req.url}. Route not found.` });
});

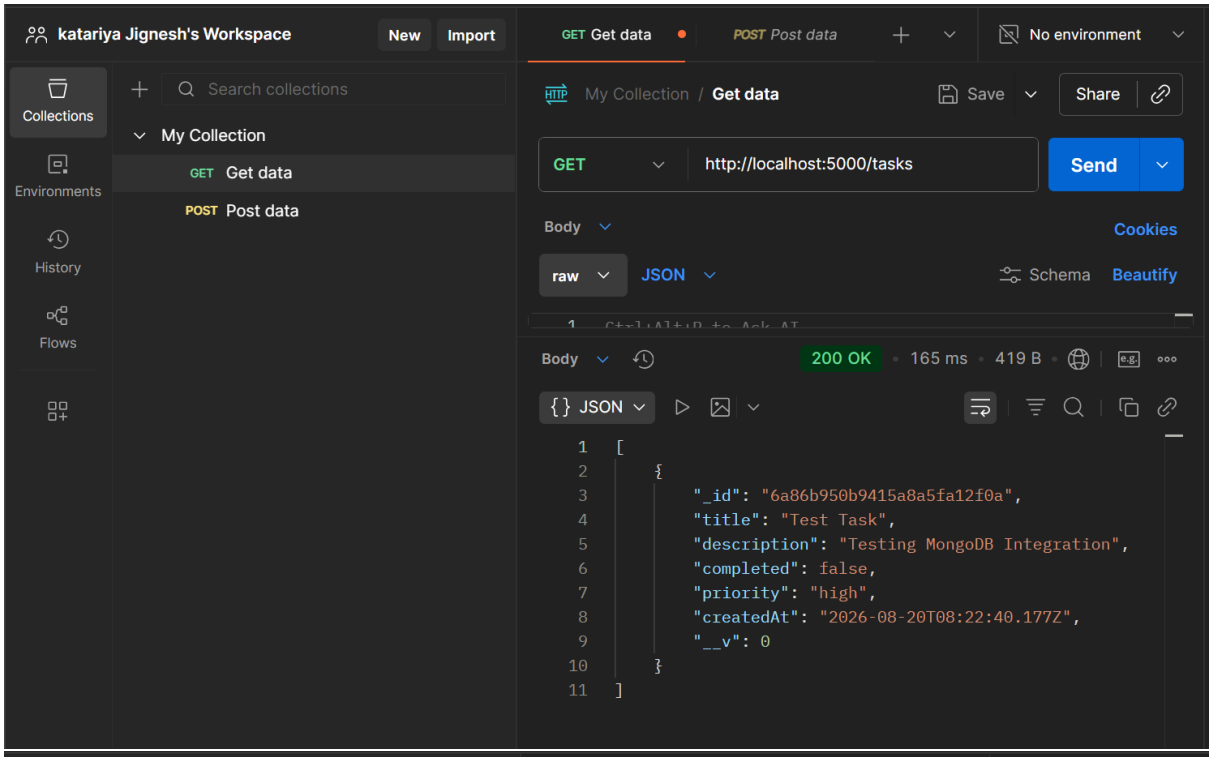
app.use((err, req, res, next) => {
  console.error("Global Error Caught:");
  console.error(err.stack || err);

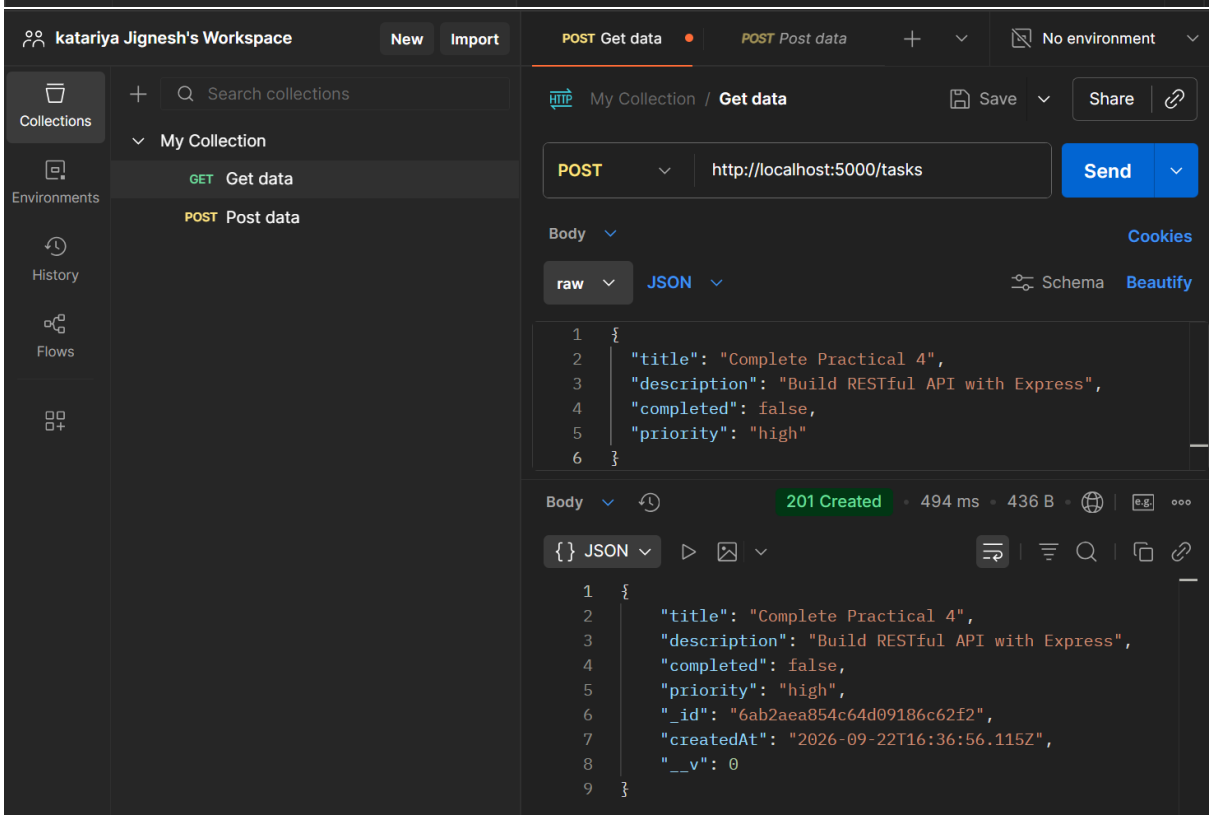
  if (err.name === 'ValidationError') {
    const details = {};
    for (let field in err.errors) {
      details[field] = err.errors[field].message;
    }
    return res.status(400).json({
      error: "Validation Error",
      message: "The request data failed schema validation.",
      details
    });
  }

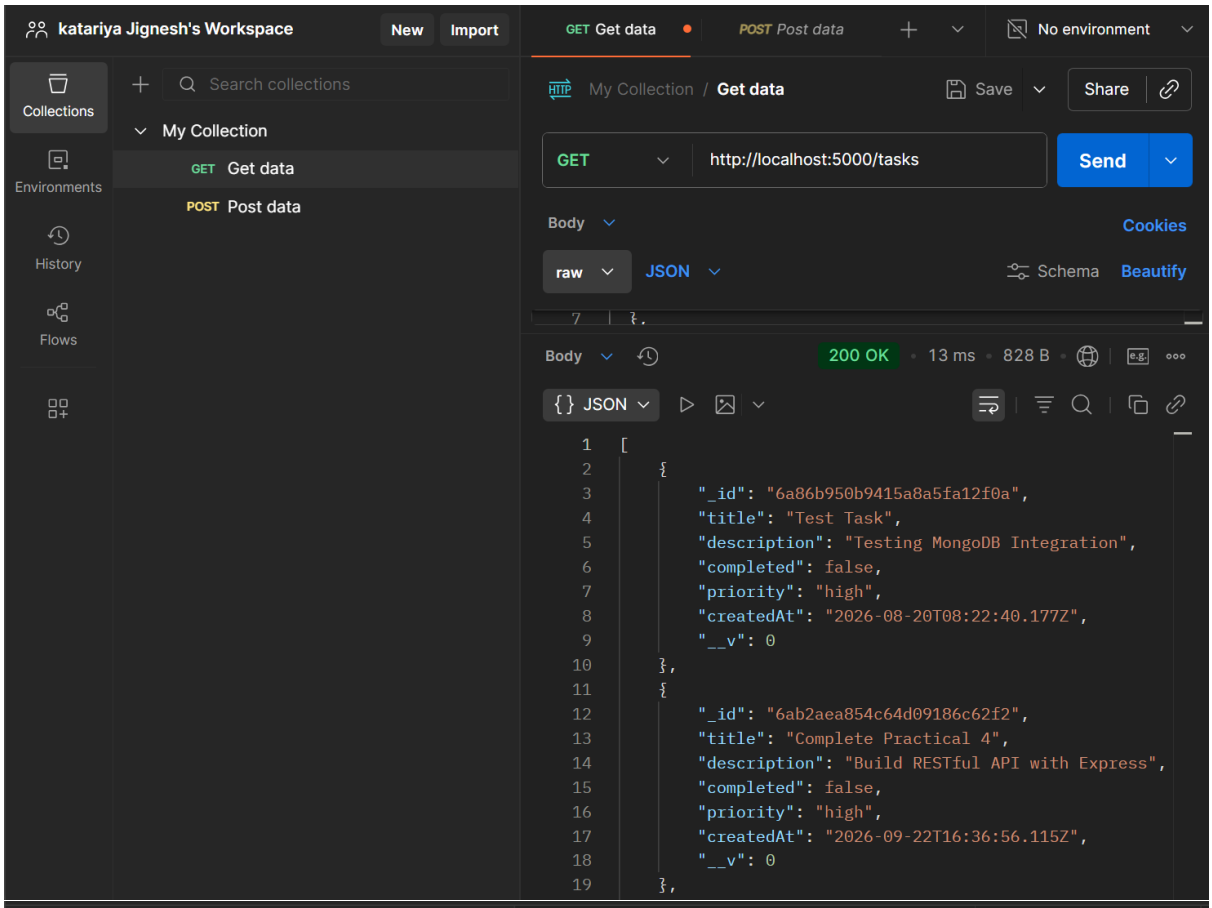
  if (err.name === 'CastError') {
    return res.status(400).json({
      error: "Bad Request",
      message: `Invalid value for field '${err.path}': ${err.value}`
    });
  }

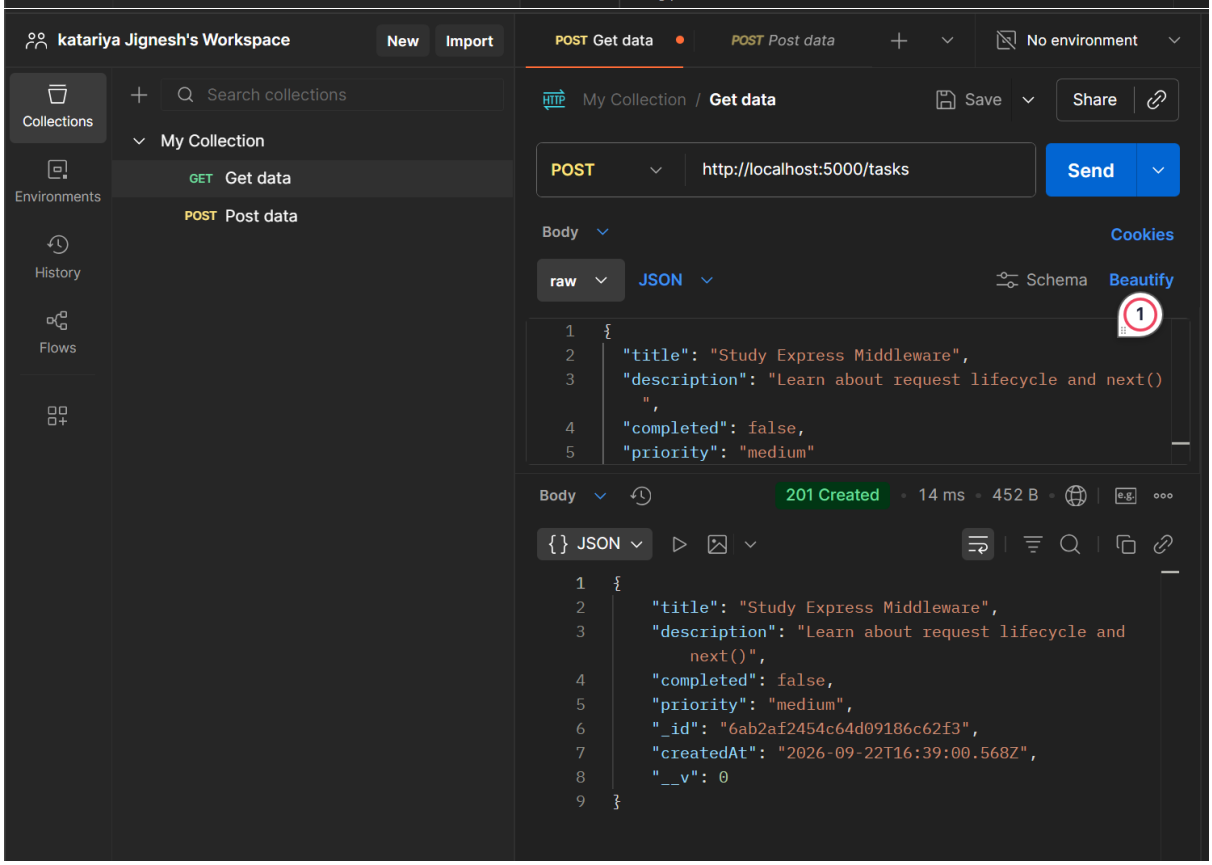
  res.status(500).json({
    error: "Something went wrong",
    message: err.message
  });
});
```

6. Test all 4 endpoints using Postman or Thunder Client, verifying correct status codes.






katariya Jignesh's Workspace
New
Import
Collections
Environments
History
Flows
Search collections
My Collection
GET Get data
POST Post data
GET Get data
POST Post data
No environment
My Collection / Get data
Save
Share
GET
http://localhost:5000/tasks
Send
Body
Cookies
raw
JSON
Schema
Beautify
Body
200 OK
13 ms
828 B
{ } JSON
[
{
"_id": "6a86b950b9415a8a5fa12f0a",
"title": "Test Task",
"description": "Testing MongoDB Integration",
"completed": false,
"priority": "high",
"createdAt": "2026-08-20T08:22:40.177Z",
"__v": 0
},
{
"_id": "6ab2aea854c64d09186c62f2",
"title": "Complete Practical 4",
"description": "Build RESTful API with Express",
"completed": false,
"priority": "high",
"createdAt": "2026-09-22T16:36:56.115Z",
"__v": 0
},


katariya Jignesh's Workspace
New
Import
Collections
Environments
History
Flows
Search collections
My Collection
GET Get data
POST Post data
POST Get data
POST Post data
No environment
My Collection / Get data
Save
Share
POST
http://localhost:5000/tasks
Send
Body
Cookies
raw
JSON
Schema
Beautify
1
{
"title": "Study Express Middleware",
"description": "Learn about request lifecycle and next()
",
"completed": false,
"priority": "medium"
Body
201 Created
14 ms
452 B
{ } JSON
{
"title": "Study Express Middleware",
"description": "Learn about request lifecycle and
next()",
"completed": false,
"priority": "medium",
"_id": "6ab2af2454c64d09186c62f3",
"createdAt": "2026-09-22T16:39:00.568Z",
"__v": 0
}

Key Questions / Analysis	**1. Why must error-handling middleware be defined last?** Because it must catch errors from all previous middleware and routes. Therefore, it should be placed at the end of the middleware chain. **2. Difference between app.use() and route-specific middleware?** • app.use() → applies middleware to multiple/all matching routes. • Route-specific middleware → runs only for a particular route. **3. Why not send raw error stack traces to the client?** Because stack traces can expose sensitive information such as file paths, source-code details, and internal server structure. This creates security risks.